Résumé des TD 2, 3 et 4

plan

- positionnement
    - le flux
    - positionnement relatif
    - Le couple absolu/relatif
    - la positionnement fixe.
    - le positionnement flottant (float)
    - positionnement `flex`
- les sélecteurs
    - mise en place d'un fichier css
    - le modèle parent-enfants
    - les premiers sélecteurs (balise, id, classe)
    - le sélecteur multiple
    - Cascade, spécificité et héritage en css
    - sélecteurs de descendance
    - sélecteurs d'adjacence
    - sélecteur d'attribut
    - Pseudo-classe et pseudo-élément

positionnement en css

le flux

Le **flux** d'un document est le comportement **naturel d'affichage des éléments** d'une page web. Les éléments se succèdent dans l'ordre où ils sont déclarés dans le code HTML.

il existe des éléments de type `block` (`h1`, `p`, `ul`, `ol`, `dl`, `table`, `blockquote`,`div` etc.) qui se placent les uns sous les autres et des éléments de type `inline` (`a`, `img`, `strong`, `abbr`, etc.) qui se placent les uns à côté des autres.

Un élément de types `bloc`:

- occupe toute la largeur de son conteneur
- permet l'attribution de marges verticales
- permet la modification de sa hauteur et largeur

Tout élément peut être "reclassé" dans la famille opposée grâce à la propriété `display`.

```
h1 {
    display : inline;
}
```

le positionnement relatif

La position relative (`position:relative`) permet de décaler un élément par rapport à la position qu'il avait dans le flux. Les éléments qui le suivent et le précèdent ne sont pas influencés par ce décalage .

En pratique, ce comportement est rarement recherché bien qu'il puisse s'avérer utile dans certains cas.

Les propriétés `top` ou `bottom` indiquent le décalage vertical à appliquer et `left` ou `right` indiquent le décalage horizontal.

```
div {
position: relative;
top: 40px; left: 40px;}
```

le couple absolu/relatif

L'élément fixé en `absolu` est retiré du flux normal et aucun espace n'est créé pour lui sur la page.

Il est ensuite positionné **par rapport à son ancêtre le plus proche positionné** (en absolue ou relatif) s'il y en a un ou par rapport au body.

La position finale de l'élément est déterminée par les valeurs de `top`, `right`, `bottom` et `left` dans son conteneur.

```
#ancetre {
    position:relative;
}

#fils {
    position : absolute;
    top :0px;
    right : 0px;
    //positionnement en haut à droite du parent ancetre positionner en
relatif
}
```

la position `fixed`

L'élément est retiré du flux normal et aucun espace n'est laissé pour l'élément.

L'élément est positionné relativement au bloc englobant initial formé par la zone d'affichage (`viewport` ou `body`)

```
header{
    position:fixed;
    top :20px;
}
```

Un dérivée de cette position est la position `sticky` : lien

le positionnment flottant

La propriété `float` indique qu'un élément doit être retiré du flux normal et doit être placé sur le côté droit ou sur le côté gauche de son conteneur. Le texte et les autres éléments en ligne (inline) entoureront alors l'élément flottant.

```
img{
    float:right;
}
```

La propriété `clear` indique si un élément peut être situé à côté d'éléments flottants qui le précèdent ou s'il doit être déplacé vers le bas pour être en dessous de ces éléments.

```
#para{
    clear:both;
    // ce paragraphe se placera en dessous des éléments flotants
}
```

le positionnement flex

Pour le positionnement en flex, un tp complet sera proposé. Je vous présente juste la base.

On commence par positionner le conteneur en `flex` :

```
section{
    display:flex;
}
```

Flexbox dispose de la propriété `flex-direction` pour indiquer la direction de l'axe principal (direction dans laquelle les enfants flexibles sont disposés) — cette propriété est égale par défaut à row;

nous pouvons contrôler la proportion des éléments flexibles dans l'espace.

```
article {
  flex: 1;
}
```

Il s'agit d'une valeur de proportion, sans unité, définissant la quantité d'espace disponible que chaque élément flex prendra le long de l'axe principal. Dans ce cas, nous donnons à chaque élément `<article>` une valeur de 1, ce qui signifie qu'ils prendront tous une portion égale de l'espace libre après le calcul du remplissage et de la marge.

```
article:nth-of-type(2){
    flex: 2;
}
```

Vous pouvez également définir une valeur minimale de taille dans la valeur flex. Modifiez comme suit vos règles `article` existantes :

```
article {
  flex: 1 200px;
}

article:nth-of-type(3) {
  flex: 2 200px;
}
```

- Chaque élément flex reçoit d'abord 200px de l'espace disponible quand c'est possible.
- Ensuite, le reste de l'espace disponible est réparti selon les unités de proportion.
- Quand ce n'est pas possible, ce sont les unités de proportions qui sont prises en compte.

Et pour en finir avec l'alignement vertical

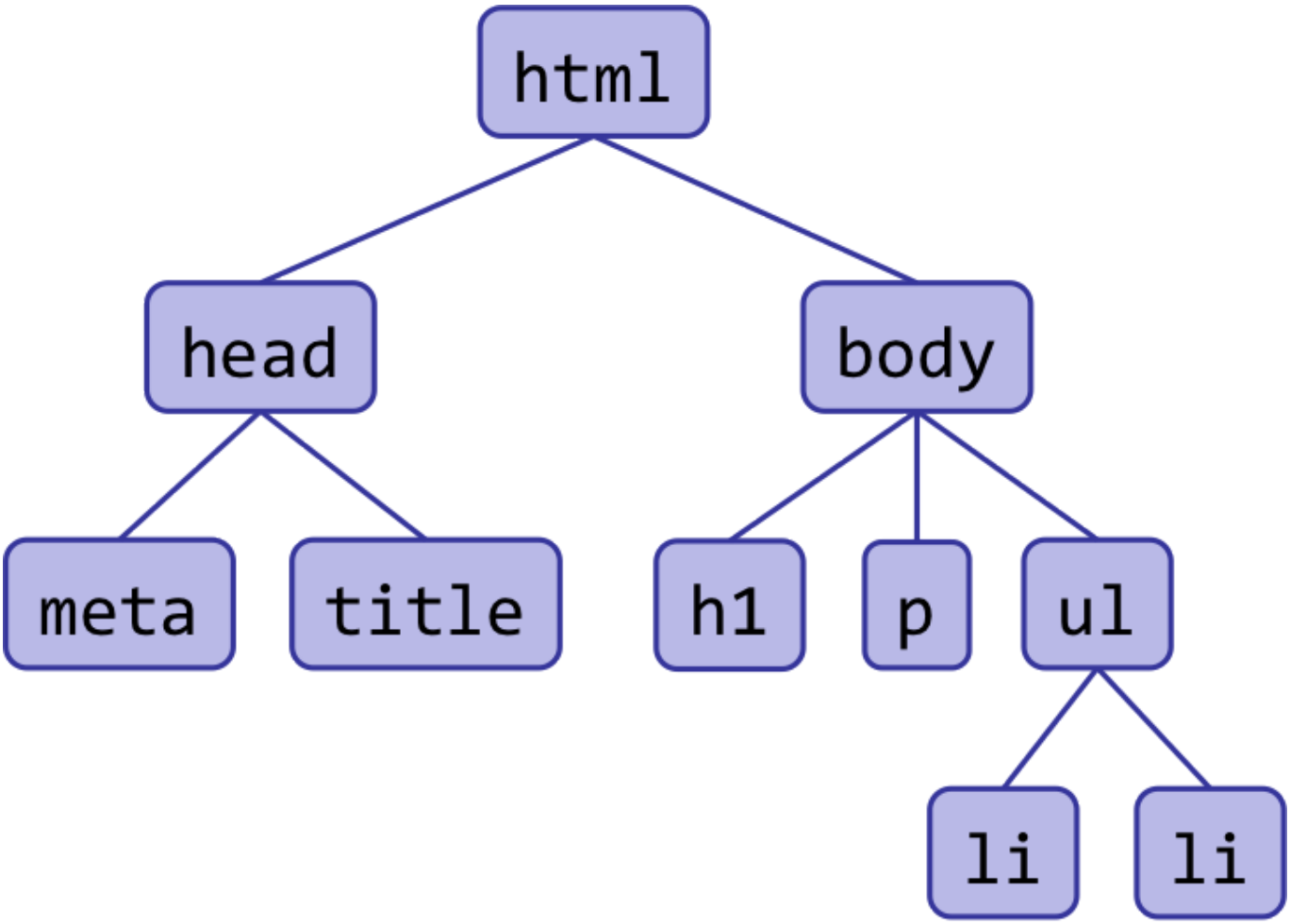
```
div {
  display: flex;
  align-items: center;              // axe secondaire
  justify-content: space-around;   //axe pricipal
}
```

les selecteurs

mise en place d'un fichier css

`<link href="monfichier.css" rel="stylesheet">` dans le `head` du document `html`.

le modèle parent-enfant



les premiers sélecteurs

- le sélecteur de balises redéfinit toutes les balises données

```
p {
    border : 2px solid red;
}
```

- le sélecteur d'id redéfinit uniquement la balise d'id donnée

```
#panel{
    display:flex;
}
```

- le sélecteur de classe rédéfinit *toutes* les balises ayant la classe donnée

```
.rouge {
    color :red;
}
```

```
<p> je suis un paragraphe</p>
<div id='panel'>
    <ul class='rouge'>
        <li>1</li>
        <li>2</li>
    </ul>
</diV>
```

le sélecteur multiple

Pour sélectionner plusieurs balises en même temps, on les sépare par des `virgules`.

```
p, article, aside {
    display:none;
}
```

un peu de vocabulaire

- `cascade` : la cascade des styles signifie que **l'ordre d'apparition des règles dans le CSS a une importance** ; quand deux règles applicables ont la même spécificité, c'est la dernière déclarée qui sera utilisée pour la mise en forme.
- `héritage` : Certaines valeurs pour une propriété CSS se transmettent des éléments parents vers leurs enfants, d'autres non. Par exemple, si vous fixez une `color` et une `font-family` pour un élément, tout élément contenu dans le premier sera mis en forme avec la même couleur et la même police, à moins qu'on lui ait appliqué directement des règles.
- `spécificité` : Quand des règles avec des sélecteurs différents s'appliquent sur un même élément, le navigateur choisit la règle qui a **la plus grande spécificité**. La spécificité mesure essentiellement combien la sélection est précise : Un sélecteur d'élément est peu spécifique : il cible tous les éléments d'un type donné dans la page —> son score est donc faible ; Un sélecteur de classe est plus spécifique — dans la page, il ne cible que les éléments dont l'attribut class a la valeur choisie —> son score est plus important. Un sélecteur d'id est encore plus spécifique, il sera donc prioritaire sur les précédents.

Sélecteurs de descendance

- l'espace permet de sélectionner toute les désecdants d'une balise :

```
header ul {
    color :red;
    //selection de toutes les balises se trouvant dans le header
}
```

- le chevron > permet de sélectionner uniquement les enfants `directs` de la balise

```
header> ul {
    color :red;
    //selection de toutes les balises se trouvant dans le header
}
```

```
<header>
    <ul>       <- enfant direct
        <li></li>
        <li>
            <ul>    <- DESCENDANT MAIS PAS ENFANT DIRECT  car dans un li
                <li></li>
                <li></li>
            </ul>
        </li>>
    </ul>
</header>
```

sélecteur d'adjacence :

- Le sélecteur adjacent (direct) désigné par le signe + permet de cibler un élément qui suit immédiatement un autre élément

l'élément suivant doit être au même niveau hiérarchique (pas à l'intérieur) que le premier élément

```
h1+p{
    color: red;
}
```

Code HTML

```
<h1> Titre </h1>
<p> oui..</p>
<p>  non..</p>
```

- Le sélecteur de voisin ~ est trop générique est sélectionne tous les voisins d'une balise données

```
h1~p{
    color: red;
}
```

Code HTML

```
<h1> Titre </h1>
<p> oui..</p>
<p>  oui..</p>
```

le sélecteur d'attribut

Avec `[]` , il est possble de sélectionner des balise ayant des attributs particuliers

```
a[target]{
    color:red;
    sélection de tous les liens ayant un attribut target
}
```

On peut également être plus précis :

```
a[target="_blanck"]
{
    color : red;
    élection de tous les liens ayant un attribut target égale à _blanck
}
```

ou plus générique :

```
a[href^="#"]{
    color:red;
    selection de tous les liens internes à la page ( commençant par #)
}
```

je vous laisse voir le cours pour plus de détail.

Pseudo-classe

Une `pseudo-classe` est un mot-clé qui peut être ajouté à un sélecteur afin d'indiquer l'état spécifique dans lequel l'élément doit être pour être ciblé par la déclaration.

La `pseudo-classe :hover`, par exemple, permettra d'appliquer une mise en forme spécifique lorsque l'utilisateur survole l'élément ciblé par le sélecteur.

```
div:hover{
    color:red;
}

a:link{
    color: black;
    text-decoration: none;
}
a:visited{
    color: #0AF;/*Bleu*/
}
a:hover{
    color: orange;
    text-decoration: underline;
}
a:active{
    background-color: #0FA;/*Vert*/
}
```

peudo-élément

Un `pseudo-élément` est un mot-clé ajouté à un sélecteur qui permet de mettre en forme certaines parties de l'élément ciblé par la règle.

```
p::first-letter {
  color: #ff0000;
  font-size: xx-large;
}
```

Deux cas particuliers : `after` et `before`

`::after` crée un pseudo-élément qui sera le dernier enfant de l'élément sélectionné. Il est souvent utilisé pour ajouter du contenu cosmétique à un élément, en utilisant la propriété CSS `content`.

```
p::after{
    content : "je rajoute ce texte à la fin de chaque paragraphe";
}
```